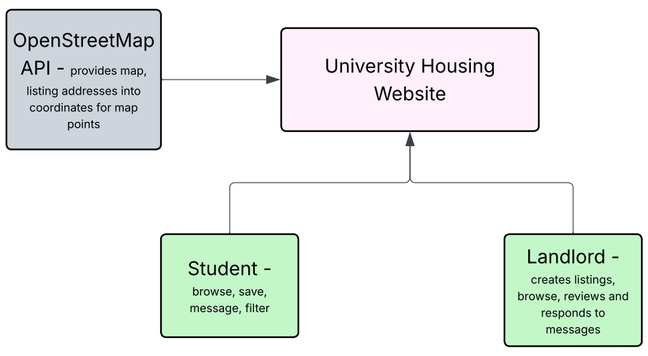
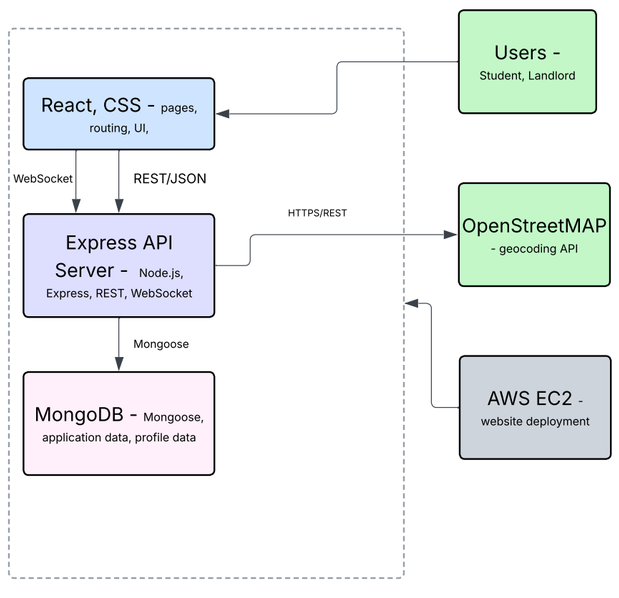
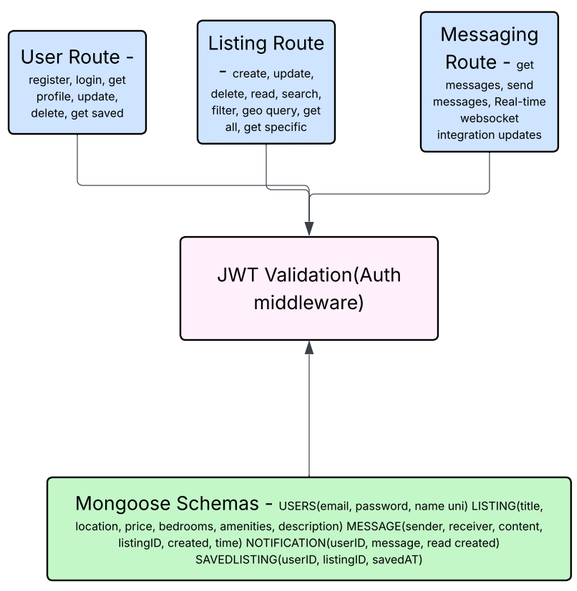
System Context - L1



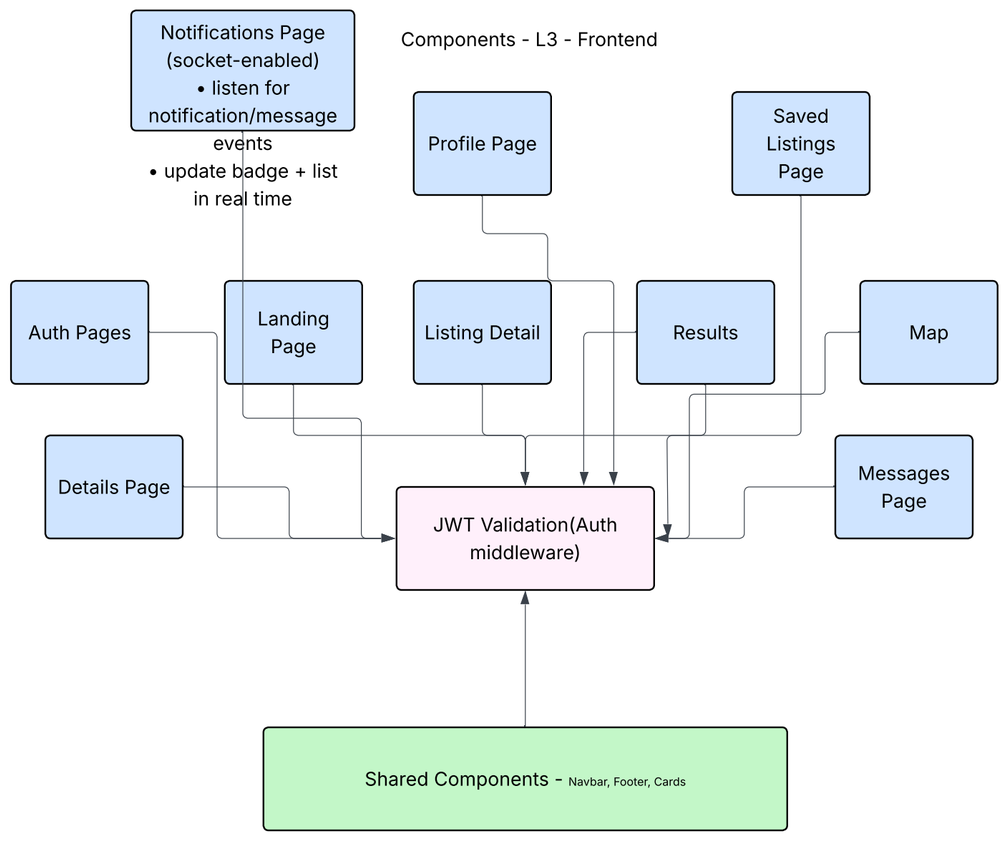
Container Diagram - L2



Components - L3 - Backend



Components - L3 - Frontend



Code Level - L4

CODE LEVEL -L4 - General

FRONT-END (React) - Browser

Landing Page
• main section introdcution
• Search bar input
• University selections
• Feature cards
• Navigate on click

Auth Pages
• Login component
• Signup component
• Form validation
• Token storage
• Error handling

Search Results Page
State: { listings: [], filters: { q, price, bedrooms, distance, ... }, loading, error }
useEffect([filters]): fetch(`/api/listings?${queryParams}`), setListings(data)
render: map listings, ListingCard + filters selections

Listing Detail Page
useEffect([id]): fetch(`/api/listings/:id`) → setListing(data)
Buttons:
• Save Listing → POST /api/saved-listings
• Message Owner → navigate to Messages page
• Apply → navigate to Application page
• View on Map → navigate to Map View

Other Pages
• Messages Page (real-time via Socket.io): GET/POST /api/messages
• Applications: POST /api/applications, GET /api/applications/me, PATCH /api/applications/:id/status
• Saved Listings: GET /api/saved-listings
• Map View: Leaflet, GET /api/listings

Notifications & Profile
Notifications:
• Real-time via Socket.io
• New matches / messages / application updates
• Mark as read (PATCH /api/notifications/:id/read)
Profile:
• User info + preferences

Shared stuff
• useOnlineStatus() + offline cache (localStorage)
• JWT in localStorage; attach Authorization: Bearer <token>; redirect on 401

BACK-END (Express + Node.js)

HTTP Server (Express) + WebSocket Server (Socket.io)
• cors(), express.json()
• JWT auth middleware on protected routes

REST API Routes (JWT)
Users:
• POST /api/users/register
• POST /api/users/login
• GET /api/users/me
• PATCH /api/users/me
• PATCH /api/users/preferences
Listings:
• GET /api/listings
• GET /api/listings/:id
• POST /api/listings
• PATCH /api/listings/:id
• DELETE /api/listings/:id
Messaging:
• GET /api/messages
• POST /api/messages
Saved Listings:
• GET /api/saved-listings
• POST /api/saved-listings
Notifications:
• GET /api/notifications
• POST /api/notifications
• PATCH /api/notifications/:id/read
Applications:
• GET /api/applications/me
• POST /api/applications
• GET /api/applications/listing/:id
• PATCH /api/applications/:id/status

WebSocket Events (Socket.io)
• connection (auth via JWT)
• socket.join('user:{userId}')
• 'message:send'
• 'message:new'
• 'notification:new'
• 'listing:updated'
• broadcast to user rooms

Request Pipeline
1) parse body 2) extract token 3) verify JWT (403 if invalid)
4) validate 5) route handler: DB queries + business logic + notifications + websocket
6) respond JSON

Mongoose ODM

fetch() + Socket.io
JSON + Authorization

MONGODB DATABASE

Collections + Indexes
• users (email, createdAt)
• listings (title, price, createdAt)
• messages (sender, receiver, createdAt)
• saved_listings (user, listing, savedAt)
• notifications (recipient, isRead, createdAt)
• applications (applicant, listing, createdAt)

Mongoose ODM
• User.findOne/findById/findByIdAndUpdate
• Listing.find with filters/text search
• Message.create + populate
• Notification.insertMany
• bcrypt + JWT signing/verification
• Geocoding calls (Nominatim)